

Taller 1

Fecha: Agosto de 2026

Indicador de logro a evaluar: Aplicar los conceptos de modelado de arquitecturas para una solución a implementar basada en microservicios

NOTAS:

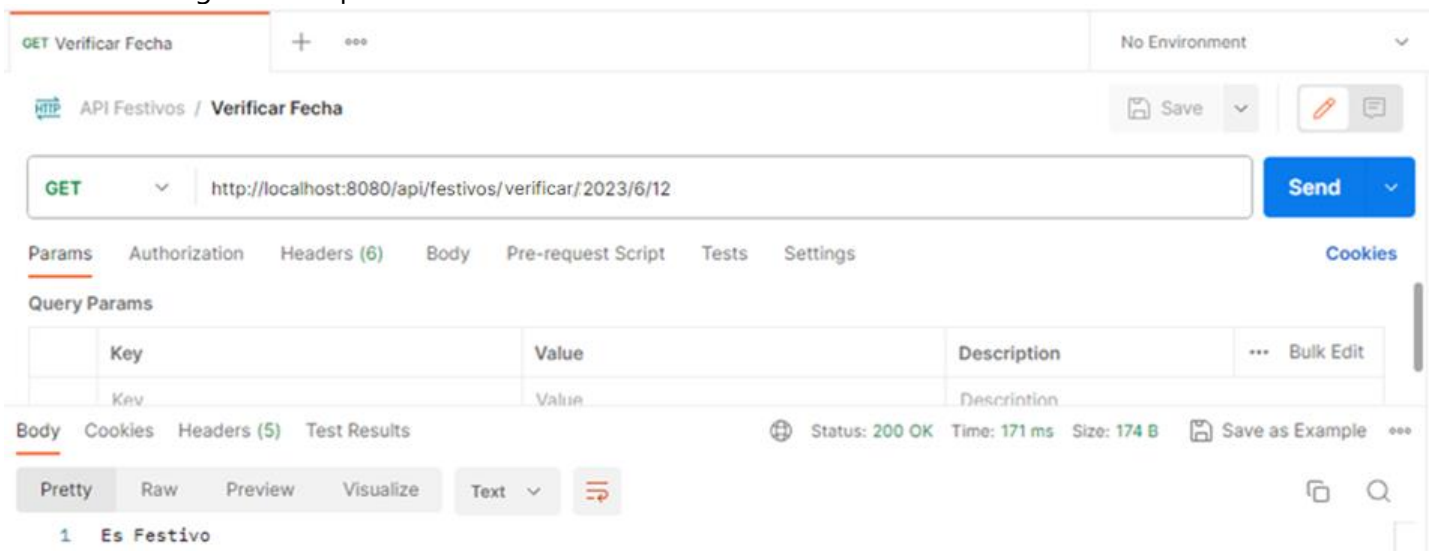
- Esta evaluación representa en total una calificación de 40% correspondiente al primero y segundo seguimiento

Usted será el encargado de implementar una solución cuya arquitectura está basada en microservicios, para lo cual debe elaborar las siguientes APIS:

1. API RESTfull en Express JS + MongoDB

Se requiere un servicio web que permita validar si una fecha determinada es festiva o no.

Por ejemplo, la ejecución del servicio para el 12 de junio de 2023, entregaría la siguiente respuesta:



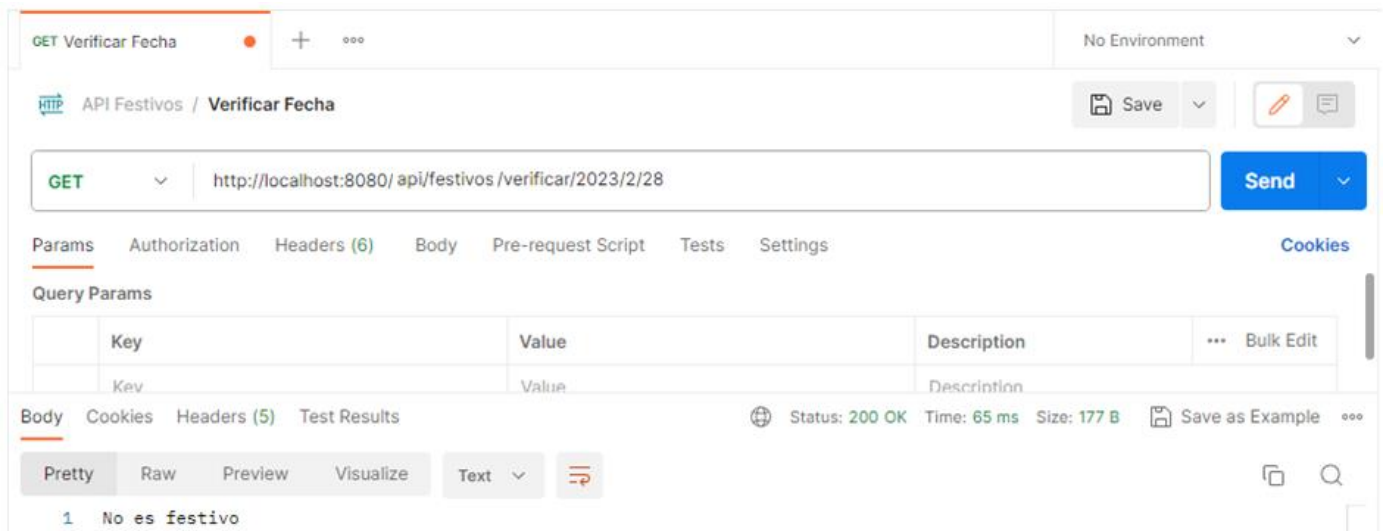
The screenshot shows a REST client interface with the following details:

- Request Method:** GET
- URL:** `http://localhost:8080/api/festivos/verificar/2023/6/12`
- Status:** 200 OK
- Time:** 171 ms
- Size:** 174 B
- Response Body:**

```
1 Es Festivo
```

Key	Value	Description
Key	Value	Description

La ejecución para el 28 de febrero de 2023:



GET Verificar Fecha

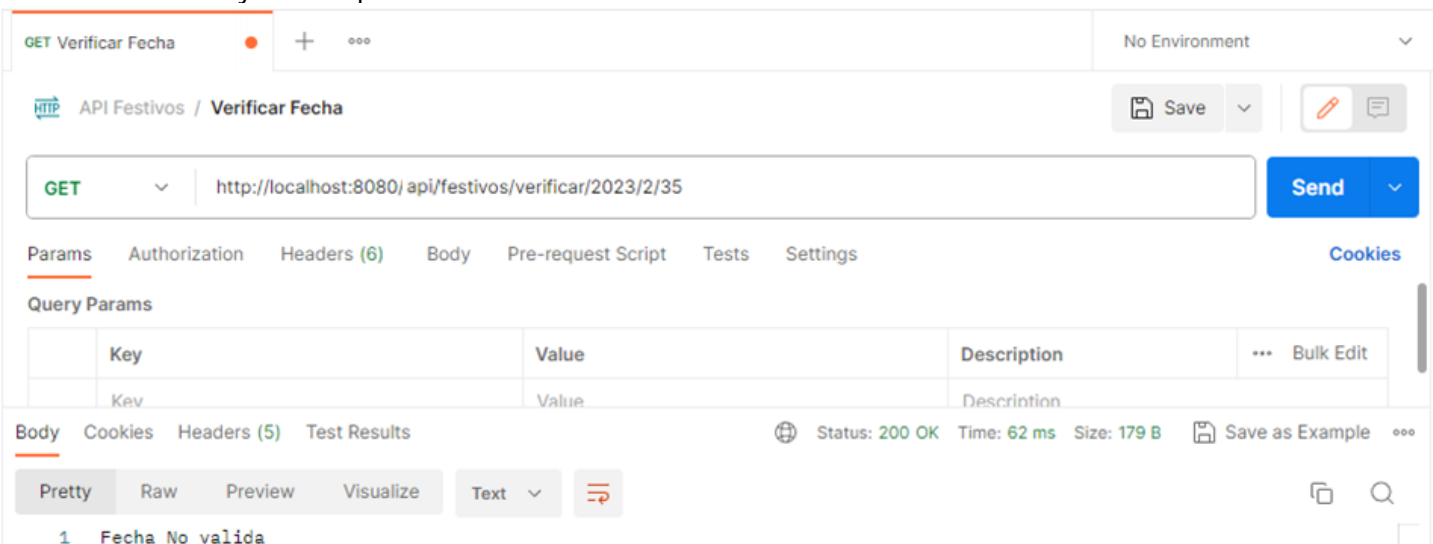
API Festivos / Verificar Fecha

GET http://localhost:8080/api/festivos/verificar/2023/2/28

Status: 200 OK Time: 65 ms Size: 177 B

1 No es festivo

Y la ejecución para una fecha no válida como el 35 de febrero de 2023:



GET Verificar Fecha

API Festivos / Verificar Fecha

GET http://localhost:8080/api/festivos/verificar/2023/2/35

Status: 200 OK Time: 62 ms Size: 179 B

1 Fecha No valida

Para ello se dispondrá de la siguiente base de datos en *MongoDB* con la lista de festivos:

Índice	Tipo	Modo de calcularlo
1	Fijo	No se puede variar

Día	Mes	Nombre	Días de Pascua
1	1	Año nuevo	
1	5	Día del Trabajo	

20	7	Independencia Colombia																																	
7	8	Batalla de Boyacá																																	
8	12	Inmaculada Concepción																																	
25	12	Navidad																																	
2	Ley de "Puente festivo"	Se traslada la fecha al siguiente lunes																																	
<table><tr><th>Día</th><th>Mes</th><th>Nombre</th><th>Días de Pascua</th></tr><tr><td>6</td><td>1</td><td>Santos Reyes</td><td></td></tr><tr><td>19</td><td>3</td><td>San José</td><td></td></tr><tr><td>29</td><td>6</td><td>San Pedro y San Pablo</td><td></td></tr><tr><td>15</td><td>8</td><td>Asunción de la Virgen</td><td></td></tr><tr><td>12</td><td>10</td><td>Día de la Raza</td><td></td></tr><tr><td>1</td><td>11</td><td>Todos los santos</td><td></td></tr><tr><td>11</td><td>11</td><td>Independencia de Cartagena</td><td></td></tr></table>				Día	Mes	Nombre	Días de Pascua	6	1	Santos Reyes		19	3	San José		29	6	San Pedro y San Pablo		15	8	Asunción de la Virgen		12	10	Día de la Raza		1	11	Todos los santos		11	11	Independencia de Cartagena	
Día	Mes	Nombre	Días de Pascua																																
6	1	Santos Reyes																																	
19	3	San José																																	
29	6	San Pedro y San Pablo																																	
15	8	Asunción de la Virgen																																	
12	10	Día de la Raza																																	
1	11	Todos los santos																																	
11	11	Independencia de Cartagena																																	
3	Basado en el domingo de pascua	La fecha se calcula obteniendo la fecha del domingo de pascua y sumándole los días que correspondan.																																	
<table><tr><th>Día</th><th>Mes</th><th>Nombre</th><th>Días de Pascua</th></tr><tr><td></td><td></td><td>Jueves Santo</td><td>-3</td></tr><tr><td></td><td></td><td>Viernes Santo</td><td>-2</td></tr><tr><td></td><td></td><td>Domingo de Pascua</td><td>0</td></tr></table>				Día	Mes	Nombre	Días de Pascua			Jueves Santo	-3			Viernes Santo	-2			Domingo de Pascua	0																
Día	Mes	Nombre	Días de Pascua																																
		Jueves Santo	-3																																
		Viernes Santo	-2																																
		Domingo de Pascua	0																																
4	Basado en el domingo de pascua y Ley de "Puente festivo"	La fecha se calcula obteniendo la fecha del domingo de pascua y sumándole los días que correspondan. La fecha calculada debe ser trasladada al siguiente lunes																																	
<table><tr><th>Día</th><th>Mes</th><th>Nombre</th><th>Días de Pascua</th></tr><tr><td></td><td></td><td>Ascensión del Señor</td><td>40</td></tr><tr><td></td><td></td><td>Corpus Christi</td><td>61</td></tr><tr><td></td><td></td><td>Sagrado Corazón de Jesús</td><td>68</td></tr></table>				Día	Mes	Nombre	Días de Pascua			Ascensión del Señor	40			Corpus Christi	61			Sagrado Corazón de Jesús	68																
Día	Mes	Nombre	Días de Pascua																																
		Ascensión del Señor	40																																
		Corpus Christi	61																																
		Sagrado Corazón de Jesús	68																																

Calculo del domingo de pascua

Para conocer el día del año que comienza la Semana Santa (es decir, cuando sería el *Domingo de Ramos*) se tiene la siguiente fórmula que calcula el número de días que deben pasar después del 15 de marzo:

$$\text{días} = d + (2b + 4c + 6d + 5) \text{ MOD } 7$$

Dónde:

$a = \text{Año MOD } 19$

$b = \text{Año MOD } 4$

$c = \text{Año MOD } 7$

$d = (19a + 24) \text{ MOD } 30$

Por ejemplo para el año 1999:

$a = 1999 \text{ MOD } 19 = 4$

$b = 1999 \text{ MOD } 4 = 3$

$c = 1999 \text{ MOD } 7 = 4$

$d = (19 \cdot 4 + 24) \text{ MOD } 30 = 10$

$\text{días} = 10 + (2 \cdot 3 + 4 \cdot 4 + 6 \cdot 10 + 5) \text{ MOD } 7 = 13$

Significa que el *Domingo de Ramos* sería el $15 + 13 = 28$ de Marzo.

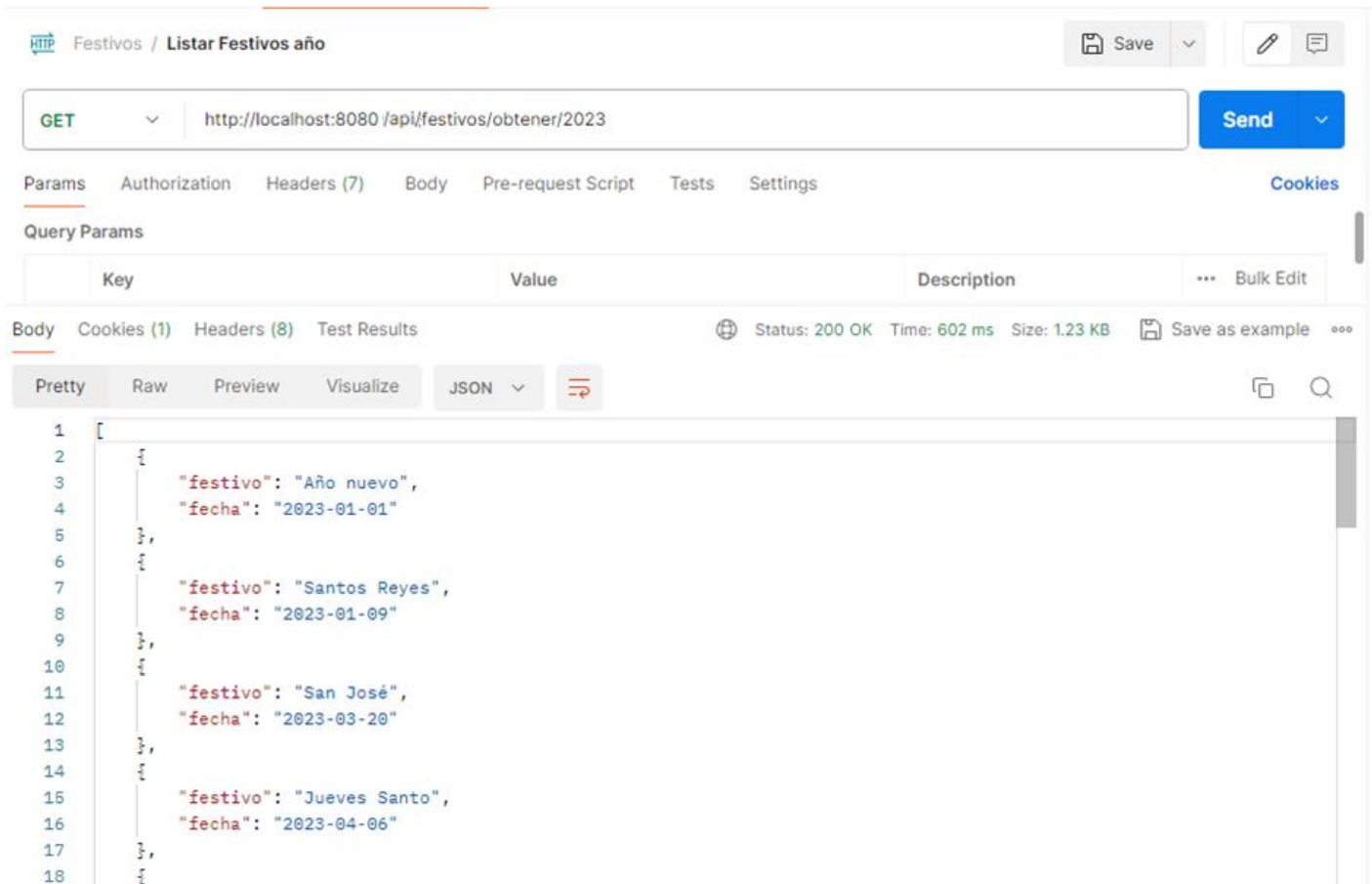
Luego el *Domingo de Pascua* sería 7 días después, o sea, Abril 4.

Marzo 1999						
L	M	M	J	V	S	D
1	2	3	4	5	6	7
8	9	10	11	12	13	14
15	16	17	18	19	20	21
22	23	24	25	26	27	28
29	30	31				

Abril 1999						
L	M	M	J	V	S	D
			1	2	3	4
5	6	7	8	9	10	11
12	13	14	15	16	17	18
19	20	21	22	23	24	25
26	27	28	29	30		

2. API RESTfull en Springboot + Postgres

Se requiere otra API que sea cliente de la API RESTfull *Festivos* (punto anterior) particularmente de un método que permita obtener la lista de festivos de un año:



HTTP Festivos / Listar Festivos año

GET http://localhost:8080/api/festivos/obtener/2023

Send

Params Authorization Headers (7) Body Pre-request Script Tests Settings Cookies

Query Params

Key	Value	Description	...	Bulk Edit
-----	-------	-------------	-----	-----------

Body Cookies (1) Headers (8) Test Results

Status: 200 OK Time: 602 ms Size: 1.23 KB Save as example

Pretty Raw Preview Visualize JSON

```
[{"festivo": "A\u00f1o nuevo", "fecha": "2023-01-01"}, {"festivo": "Santos Reyes", "fecha": "2023-01-09"}, {"festivo": "San Jos\u00e9", "fecha": "2023-03-20"}, {"festivo": "Jueves Santo", "fecha": "2023-04-06"}]
```

Adicionalmente, Implementar un m\u00e9todo web que, dado un a\u00f1o, almacene en una base de datos la lista de todos los d\u00edas del a\u00f1o clasific\u00e1ndolos as\u00ed:

- D\u00eda laboral (lunes a viernes no festivo)
- Fin de Semana (s\u00e1bado y domingo no festivo)
- D\u00eda festivo (con base en la lista obtenida de la API anterior)

Debe retornar un booleano indicando que el proceso fue completado con \u00e9xito

API Calendario / Generar Calendario

GET http://localhost:8080/api/calendario/generar/2023

Send

Params Authorization Headers (7) Body Pre-request Script Tests Settings Cookies

Query Params

Key	Value	Description	...	Bulk Edit
Key	Value	Description		

Body Cookies (1) Headers (5) Test Results Status: 200 OK Time: 856 ms Size: 168 B Save as example

Pretty Raw Preview Visualize JSON

```
1 true
```

Por último. Implementar un método web que, dado el año, retorne el calendario completo con los días clasificados

http://localhost:8080/api/calendario/listar/2023

GET http://localhost:8080/api/calendario/listar/2023

Send

Params Authorization Headers (7) Body Pre-request Script Tests Settings Cookies

Query Params

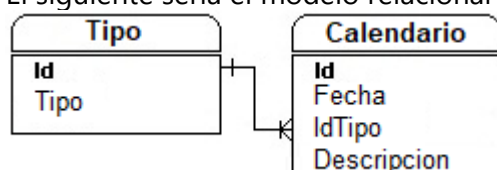
Key	Value	Description	...	Bulk Edit

Body Cookies (1) Headers (5) Test Results Status: 200 OK Time: 280 ms Size: 40.5 KB Save as example

Pretty Raw Preview Visualize JSON

```
1 [
2   {
3     "id": 1,
4     "fecha": "2023-01-01T05:00:00.000+00:00",
5     "tipo": {
6       "id": 3,
7       "tipo": "Día festivo "
8     },
9     "descripcion": "Domingo"
10  },
11  {
12    "id": 2,
13    "fecha": "2023-01-02T05:00:00.000+00:00",
14    "tipo": {
15      "id": 1,
16      "tipo": "Día laboral"
17    },
18    "descripcion": "Lunes"
```

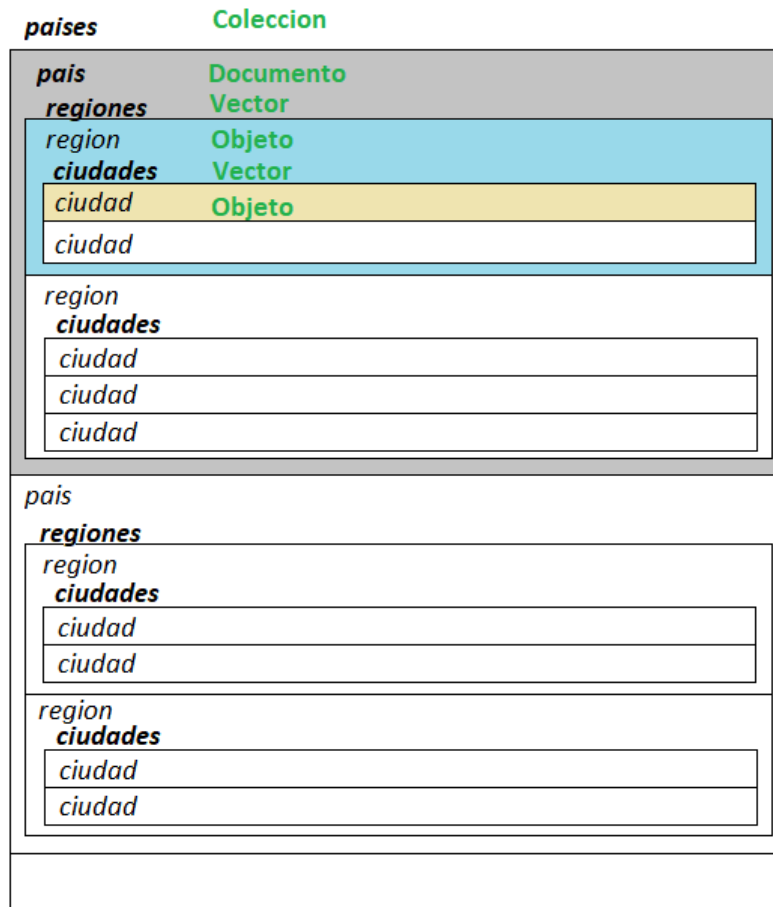
El siguiente sería el modelo relacional para la base de datos:



Se le comisiona entregar los diagramas de arquitectura para las 2 APIs anteriores

A modo de ejemplo, se modelarán a continuación 2 APIs para otra solución informática basada en microservicios

1. Una API REST que permita realizar las operaciones CRUD para todos los objetos de una base de datos NoSql para registrar información de División Política Mundial, de acuerdo al siguiente modelo:



Ejecutando la aplicación, se pueden realizar las siguientes solicitudes a la API:

- Obtener la lista de países:

GET ▼ http://localhost:3030/paises Send ▼

Params Authorization Headers (7) **Body** Pre-request Script Tests Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON ▼ Beautify

1

Body Cookies (1) Headers (7) Test Results ⊕ Status: 200 OK Time: 16 ms Size: 36.56 KB Save Response ▼

Pretty Raw Preview Visualize JSON ▼ ⋮

```
1 {
2   {
3     "_id": "6336235e59523347605a2fcc",
4     "id": 4,
5     "nombre": "Afganistán",
6     "continente": "ASIA",
7     "tipoRegion": "ESTADO",
8     "codigoAlfa2": "AF",
9     "codigoAlfa3": "AFG"
10  },
11  {
12    "_id": "6336235e59523347605a2fcd",
13    "id": 8,
14    "nombre": "Albania",
15    "continente": "EUROPA",
16    "tipoRegion": "ESTADO",
17    "codigoAlfa2": "AL",
18    "codigoAlfa3": "ALB"
```

- Agregar un nuevo país:

POST ▼ http://localhost:3030/paises/agregar Send ▼

Params Authorization Headers (9) **Body** Pre-request Script Tests Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON ▼ Beautify

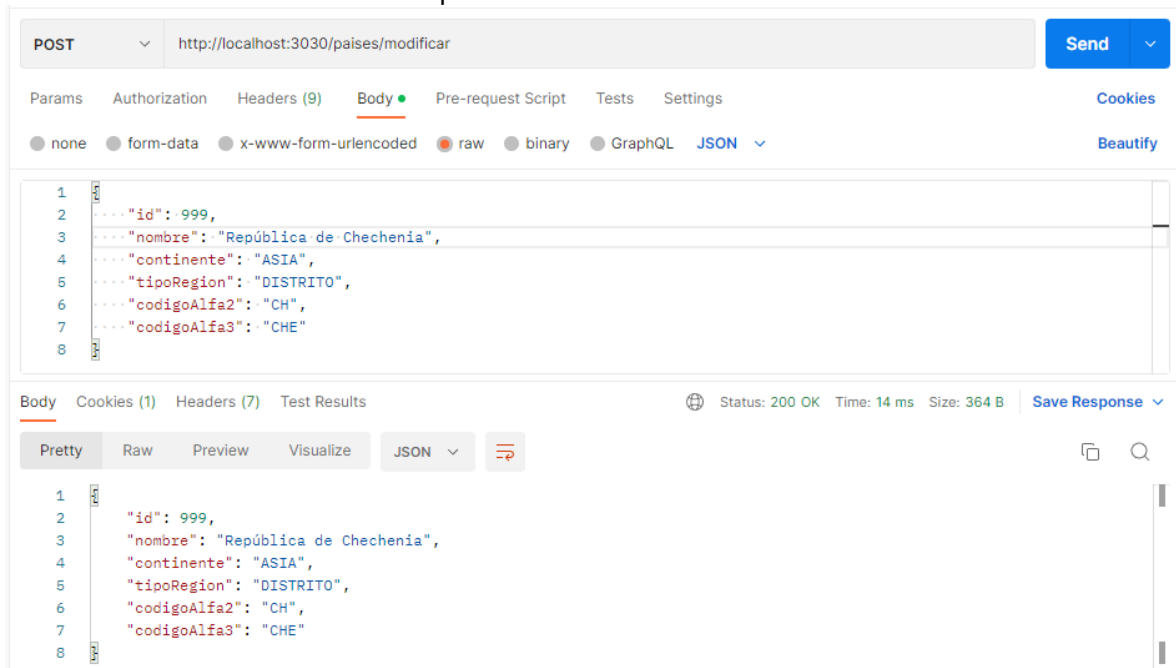
```
1 {
2   "id": 999,
3   "nombre": "Chechenia",
4   "continente": "ASIA",
5   "tipoRegion": "DISTRITO",
6   "codigoAlfa2": "CH",
7   "codigoAlfa3": "CHE"
8 }
```

Body Cookies (1) Headers (7) Test Results ⊕ Status: 200 OK Time: 1829 ms Size: 350 B Save Response ▼

Pretty Raw Preview Visualize JSON ▼ ⋮

```
1 {
2   "id": 999,
3   "nombre": "Chechenia",
4   "continente": "ASIA",
5   "tipoRegion": "DISTRITO",
6   "codigoAlfa2": "CH",
7   "codigoAlfa3": "CHE"
8 }
```

- Modificar los datos de un país:



The screenshot shows a REST client interface with a POST request to `http://localhost:3030/paises/modificar`. The request body is a JSON object with the following fields: `id` (999), `nombre` (República de Chechenia), `continente` (ASIA), `tipoRegion` (DISTRITO), `codigoAlfa2` (CH), and `codigoAlfa3` (CHE). The response status is 200 OK, with a time of 14 ms and a size of 364 B. The response body is a JSON object with the same fields as the request body.

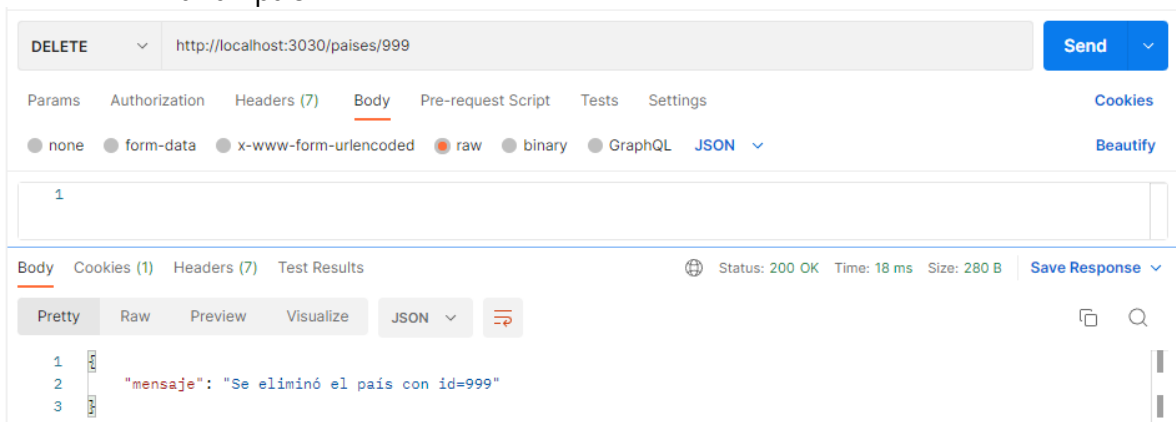
```
POST http://localhost:3030/paises/modificar

{
  "id": 999,
  "nombre": "República de Chechenia",
  "continente": "ASIA",
  "tipoRegion": "DISTRITO",
  "codigoAlfa2": "CH",
  "codigoAlfa3": "CHE"
}
```

Body Cookies (1) Headers (7) Test Results Status: 200 OK Time: 14 ms Size: 364 B Save Response

```
1 {
2   "id": 999,
3   "nombre": "República de Chechenia",
4   "continente": "ASIA",
5   "tipoRegion": "DISTRITO",
6   "codigoAlfa2": "CH",
7   "codigoAlfa3": "CHE"
8 }
```

- Eliminar un país



The screenshot shows a REST client interface with a DELETE request to `http://localhost:3030/paises/999`. The response status is 200 OK, with a time of 18 ms and a size of 280 B. The response body is a JSON object with the field `mensaje` (Se eliminó el país con id=999).

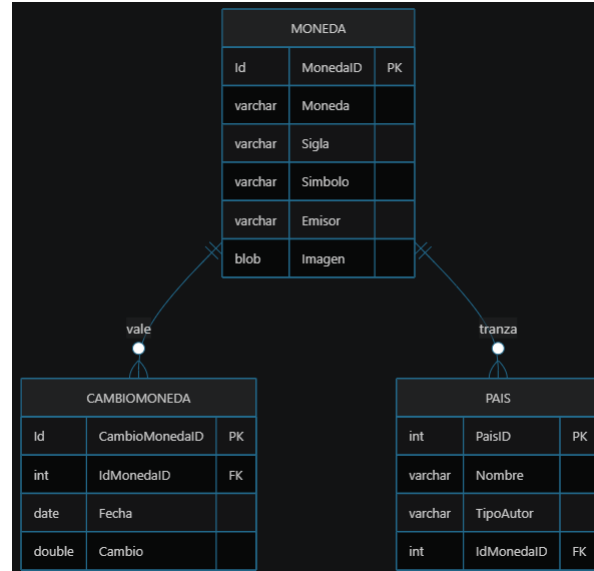
```
DELETE http://localhost:3030/paises/999

1
```

Body Cookies (1) Headers (7) Test Results Status: 200 OK Time: 18 ms Size: 280 B Save Response

```
1 {
2   "mensaje": "Se eliminó el país con id=999"
3 }
```

2. El modelo relacional de una base de datos para registro de las monedas del mundo y sus respectivos cambios con respecto al dólar, es el siguiente:



Se desea una API que permita realizar las siguientes operaciones:

- Buscar la moneda de un país

API Monedas / Buscar Moneda por Pais

GET Send

Params Authorization Headers (8) Body Pre-request Script Tests Settings Cookies

Type Bearer T... Token

The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization.

Token: eyJhbGciOiJIUzI1NiJ9.eyJzdWUiOiJmcmF5b3NvcmlvliwiaWF0IjoxNzEzMDQwMTE5LCJleHAIjE3MTMwNDE5MTF9.i-oTszEfhUx-keAQeTCjC4n4DJQukkKI50QaojUzQQ

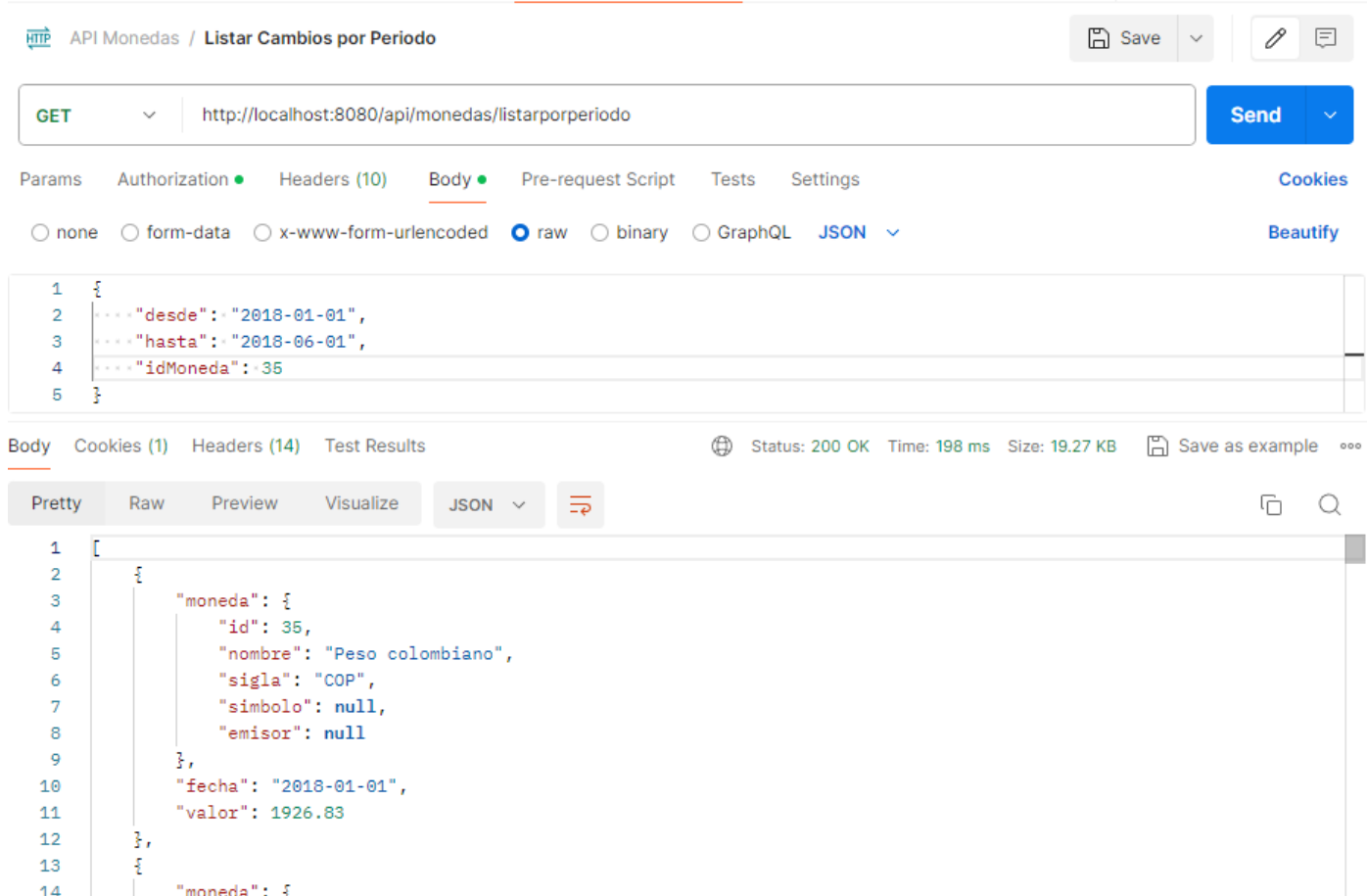
Body Cookies (1) Headers (14) Test Results Status: 200 OK Time: 199 ms Size: 502 B Save as example

Pretty Raw Preview Visualize JSON

```

1 {
2   "id": 35,
3   "nombre": "Peso colombiano",
4   "sigla": "COP",
5   "simbolo": null,
6   "emisor": null
7 }
  
```

- Listar los cambios que ha tenido una moneda en un rango de fechas



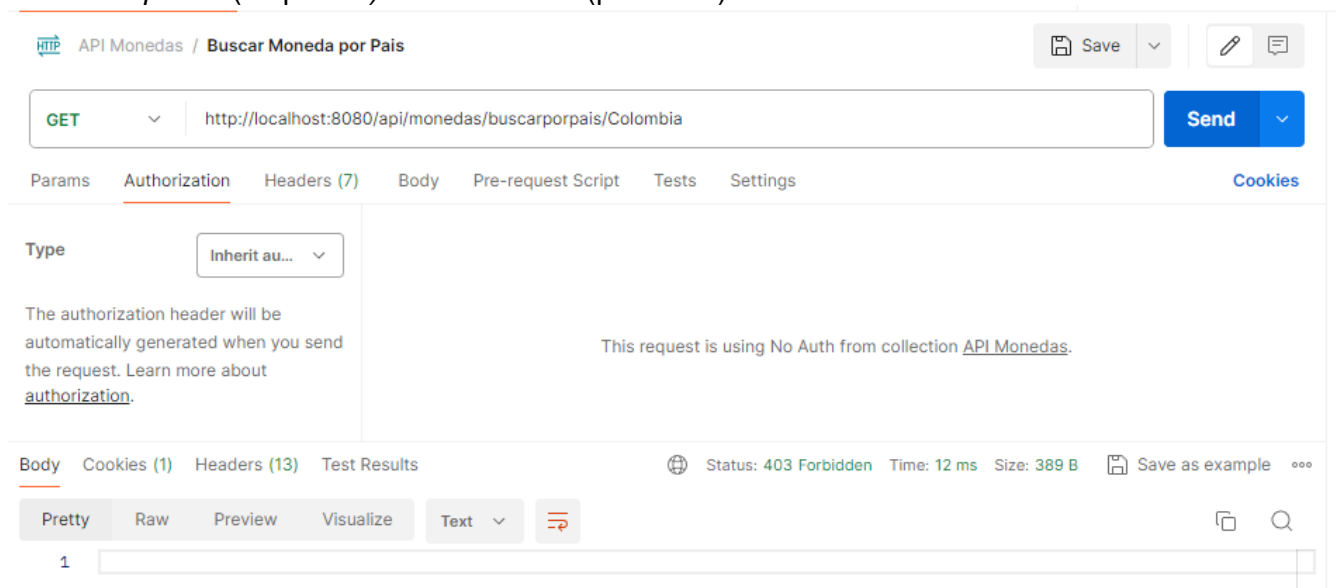
The screenshot shows a Postman interface for a collection named "API Monedas". The selected request is "Listar Cambios por Periodo" with a GET method. The URL is "http://localhost:8080/api/monedas/listarporperiodo". The request body is a JSON object:

```
1 {
2   "desde": "2018-01-01",
3   "hasta": "2018-06-01",
4   "idMoneda": 35
5 }
```

The response status is 200 OK. The response body is a JSON array:

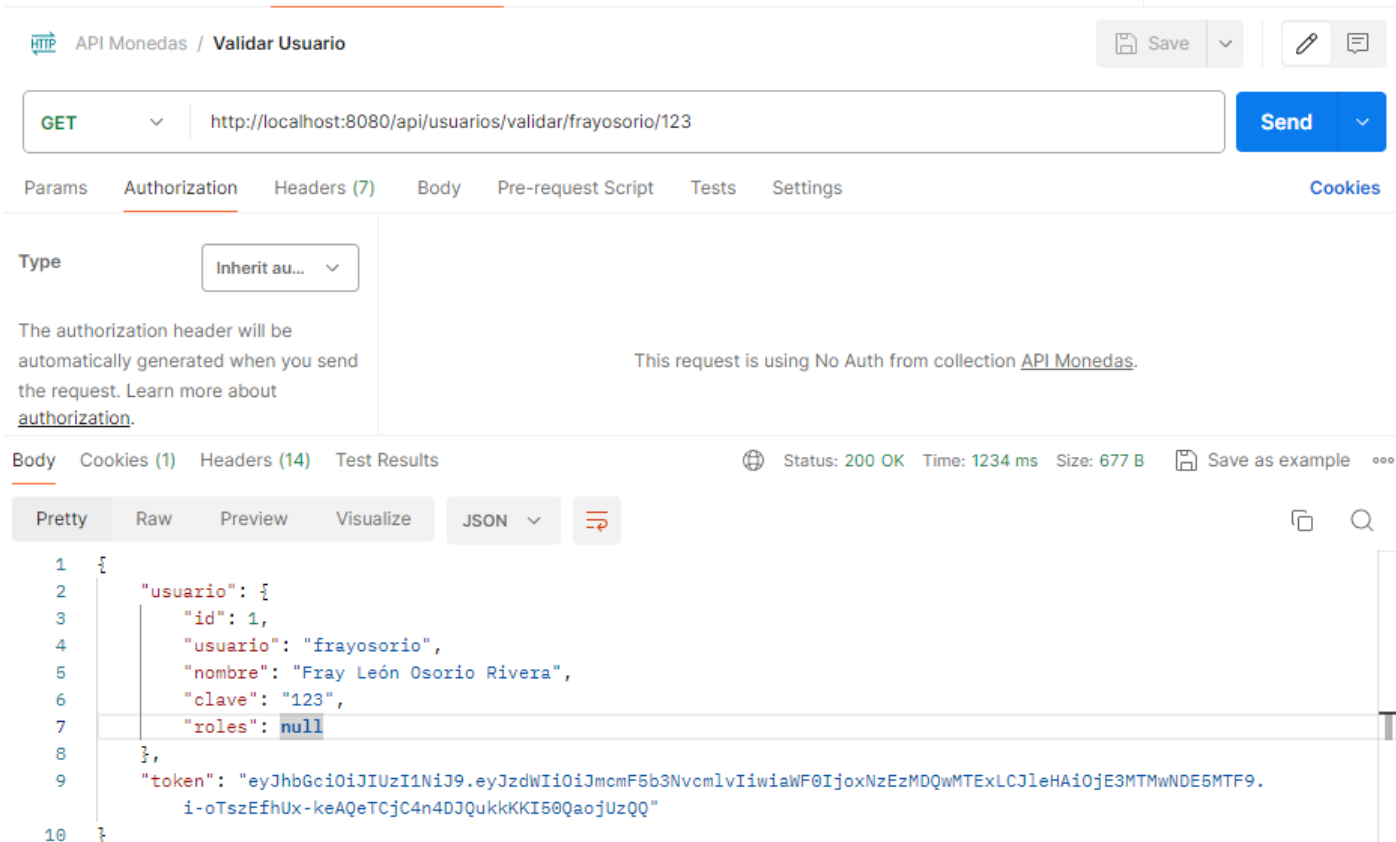
```
1 [
2   {
3     "moneda": {
4       "id": 35,
5       "nombre": "Peso colombiano",
6       "sigla": "COP",
7       "simbolo": null,
8       "emisor": null
9     },
10    "fecha": "2018-01-01",
11    "valor": 1926.83
12  },
13  {
14    "moneda": {
```

Como se puede observar en los *request* (solicitudes), se debe proporcionar en ellas el respectivo token de seguridad, que, en caso de no hacerse, se obtendrá un *response* (respuesta) **403 Forbidden** (prohibido):



The screenshot shows a Postman interface for a collection named "API Monedas". The selected request is "Buscar Moneda por Pais" with a GET method. The URL is "http://localhost:8080/api/monedas/buscarporpais/Colombia". The request is configured with "No Auth" (No authentication). The response status is 403 Forbidden. The response body is empty.

- Por ello, se debe realizar también un método web que permita validar las credenciales de usuario (*login*) y devuelva tanto el objeto del usuario correspondiente como el token de seguridad aceptado:



The screenshot shows a REST client interface with the following details:

- URL:** `http://localhost:8080/api/usuarios/validar/frayosorio/123`
- Method:** GET
- Status:** 200 OK
- Time:** 1234 ms
- Size:** 677 B
- Body (JSON):**

```
{
  "usuario": {
    "id": 1,
    "usuario": "frayosorio",
    "nombre": "Fray León Osorio Rivera",
    "clave": "123",
    "roles": null
  },
  "token": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJmcmF5b3NvcmlvIiwiaWF0IjoxNzEzMDQwMTExLCJleHAiOiE3MTMwNDU5MTF9.1i-oTszEfUx-keAQeTCjC4n4DJQukkKI50QaojUzQQ"
```

A continuación, se muestran los Diagramas de Arquitectura de este backend los cuales están escritos en **Mermaid**¹.

En primer lugar, para la API de *División Política* se tendrá un **Diagrama de Arquitectura por Capas** el cual es una representación gráfica que organiza los componentes de un sistema de software en niveles jerárquicos horizontales (layers), donde cada capa tiene una responsabilidad única y bien definida dentro del sistema (principio de separación de conceptos o *Separation of Concerns*).

En esta arquitectura, la regla fundamental es que cada capa solo interactúa con la capa inmediatamente inferior (o superior, al responder), evitando dependencias cruzadas entre componentes distantes.

Mermaid (o **Mermaid.js**) es un lenguaje de marcado basado en texto (*Diagrams as Code*) que permite definir diagramas de flujo, arquitectura, clases, secuencia y el modelo C4 mediante código simple, el cual luego es interpretado y renderizado gráficamente por herramientas como VS Code, GitHub, Notion o el editor en línea Mermaid Live

Para poder dibujar esta arquitectura, utilizaremos las siguientes instrucciones:

graph TD / flowchart TD	Inicializa el diagrama. graph o flowchart definen el tipo de gráfico, y TD (Top-Down) establece la orientación de arriba hacia abajo (también se puede usar LR para izquierda a derecha).
subgraph Nombre [Título] ... end	Agrupa varios nodos dentro de una caja visual para representar una capa, contenedor o módulo.
[Texto]	Define un nodo rectangular estándar (ej. Controllers[Controladores]).
[(Texto)]	Define un nodo con forma de cilindro, utilizado convencionalmente para bases de datos o almacenamiento.
-->	Dibuja una flecha continua unidireccional para representar el flujo principal o llamadas a métodos/peticiones.
.->	Dibuja una flecha punteada, utilizada convencionalmente para representar retornos de datos, respuestas HTTP o interfaces de implementación.
--> Texto 	Agrega una etiqueta explicativa sobre la flecha para detallar la acción o el protocolo (ej. --> Petición HTTP).

El siguiente sería el código:

```
graph TD
    %% Cliente / Capa de Presentación Externa
    subgraph ClientLayer [Capa de Cliente]
        Client[Cliente Web / Móvil / Postman / Swagger UI]
    end

    %% Capa de Entrada y Enrutamiento (API Gateway / Routes)
    subgraph PresentationLayer [Capa de Presentación / API]
        Index[index.js / app.js]
        Routes[Rutas Express<br/><i>pais.rutas.js, region.rutas.js</i>]
        Validators[Middlewares / Validadores<br/><i>pais.validador.js, region.validador.js, ciudad.validador.js</i>]
    end

    %% Capa de Lógica de Negocio (Controladores)
    subgraph BusinessLayer [Capa de Lógica de Negocio]
        Controllers[Controladores<br/><i>pais.controlador.js, region.controlador.js</i>]
    end
```

```
%% Capa de Acceso a Datos (Repositorios / Modelos)
subgraph DataAccessLayer [Capa de Acceso a Datos]
    Repositories[Repositorios]
    Modelos<br/><i>pais.repositorio.js</i>
end

%% Capa de Persistencia
subgraph PersistenceLayer [Capa de Persistencia]
    DB[(Base de Datos - MongoDB)]
end

%% Flujo de la Petición (Request)
Client -->|1. Petición HTTP| Index
Index -->|2. Delega a| Routes
Routes -->|3. Valida datos| Validators
Validators -->|4. Pasa filtro| Controllers
Controllers -->|5. Solicita operación| Repositories
Repositories -->|6. Consulta / Modifica| DB

%% Flujo de la Respuesta (Response)
DB -.->|7. Retorna datos| Repositories
Repositories -.->|8. Procesa resultado| Controllers
Controllers -.->|9. Respuesta JSON| Client

%% Estilos de Nodos
style ClientLayer fill:#e1f5fe,stroke:#0288d1,stroke-width:2px
style PresentationLayer fill:#fff3e0,stroke:#f57c00,stroke-width:2px
style BusinessLayer fill:#e8f5e9,stroke:#388e3c,stroke-width:2px
style DataAccessLayer fill:#f3e5f5,stroke:#7b1fa2,stroke-width:2px
style PersistenceLayer fill:#ffebee,stroke:#d32f2f,stroke-width:2px
```

Con la respectiva visualización:

